>_ saas-seguro · by @andrevale.dev

Relatório de segurança

clinica-facil (app de exemplo) · 19 de outubro de 2026 às 16:12

Next.js | Supabase | Prisma | Stripe | 12 arquivos analisados | revisado por IA

0
de 100

Nota de segurança: Crítico

29 brecha(s) confirmada(s) no código.

7	11	7	4
CRÍTICO	ALTO	MÉDIO	BAIXO

Checklist de 44 pontos: **6 aprovados** · 28 com falha · 8 para conferir manualmente · 2 não se aplicam.

Corrija primeiro 18 ITEM(NS)

Brechas críticas e altas, na ordem em que devem ser corrigidas.

CRÍTICO	SS-01	Chave da OpenAI escrita no código	src/lib/ai.ts:2
CRÍTICO	SS-02	Segredo em variável pública ou em código que roda no navegador	.env:4
CRÍTICO	SS-03	Senha guardada ou comparada sem hash forte	src/app/api/login/route.ts:6
CRÍTICO	SS-04	Consultas por ID sem conferir o dono do registro	src/app/api/pacientes/[id]/route.ts:3
CRÍTICO	SS-05	Tabelas do Supabase sem Row Level Security	supabase/migrations/001_init.sql:1
CRÍTICO	SS-06	Webhook de pagamento sem verificação de assinatura	src/app/api/webhooks/stripe/route.ts:2
CRÍTICO	SS-07	Valor da cobrança vindo do front	src/app/api/checkout/route.ts:6
ALTO	SS-08	Um arquivo .env já foi commitado no passado	.env

ALTO	SS-09	Token de segurança gerado com Math.random	src/app/api/login/route.ts:9
ALTO	SS-10	Rotas que parecem não exigir login	src/app/api/checkout/route.ts:3
ALTO	SS-11	Token de sessão guardado no localStorage	src/components/Login.tsx:2
ALTO	SS-12	Login sem limite de tentativas	src/app/api/login/route.ts
ALTO	SS-13	Entradas sem validação no backend	src/app/api/checkout/route.ts:4
ALTO	SS-14	Porta do banco publicada no docker-compose	docker-compose.yml:5
ALTO	SS-15	URLs sem HTTPS na configuração	src/app/api/checkout/route.ts:7
ALTO	SS-16	Stack trace ou modo debug exposto ao usuário	src/app/api/gerar/route.ts:9
ALTO	SS-17	Rotas que chamam IA paga sem cota por usuário	src/app/api/gerar/route.ts:1
ALTO	SS-18	Dados sensíveis sem criptografia de campo	prisma.schema.prisma:8

Achados detalhados 29 NO TOTAL

Cada achado traz onde está, por que é perigoso, o passo a passo e um prompt pronto para corrigir.

SEGREDOS E CHAVES

CRÍTICO SS-01 · item 1.1

Chave da OpenAI escrita no código

1 ocorrência(s) de chave da openai em arquivos do projeto.

ONDE ESTÁ

```
src/lib/ai.ts:2
export const openai = new OpenAI({ apiKey: "sk-pr…[redigido]" });
```

POR QUE É PERIGOSO

Quem tiver acesso ao repositório usa a chave da OpenAI do projeto e a fatura vem para você.

PASSO A PASSO PARA CORRIGIR

1. Revogue a chave no painel da OpenAI (platform.openai.com → API keys) e gere uma nova.
2. Em src/lib/ai.ts, troque a chave escrita por process.env.OPENAI_API_KEY.
3. Adicione OPENAI_API_KEY ao .env local e às variáveis de ambiente da Vercel.
4. Confirme que nenhuma outra chave sobrou no código rodando a varredura de novo.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Em src/lib/ai.ts, remova a chave da OpenAI escrita no código e leia de process.env.OPENAI_API_KEY, falhando com erro claro se a variável não existir. Adicione OPENAI_API_KEY ao .env.example (sem valor). Mostre o diff antes de commitar.

CRÍTICO SS-02 · item 1.4

Segredo em variável pública ou em código que roda no navegador

Essas variáveis são embutidas no JavaScript entregue ao visitante.

ONDE ESTÁ

```
.env:4
NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY=eyJhb…[redigido]
```

POR QUE É PERIGOSO

A service role do Supabase ignora a RLS. Com o prefixo NEXT_PUBLIC_, ela vai para o navegador e qualquer visitante lê e apaga todo o banco.

PASSO A PASSO PARA CORRIGIR

1. Gere uma nova service role no painel do Supabase (Settings → API → Reset) e trate a antiga como vazada.
2. Renomeie a variável para SUPABASE_SERVICE_ROLE_KEY (sem NEXT_PUBLIC_) no .env e na Vercel.
3. Use a service role só em rotas de servidor; no front, use apenas a anon key.
4. Rode a varredura de novo para confirmar.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Procure todos os usos de NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY no projeto. Renomeie para SUPABASE_SERVICE_ROLE_KEY e garanta que ela só seja usada em código de servidor (rotas de API). No front, use apenas a anon key. Mostre o diff antes de commitar.

ALTO SS-08 · item 1.3

Um arquivo .env já foi commitado no passado

As chaves que estavam nesse .env continuam acessíveis no histórico.

ONDE ESTÁ

```
.env
adicionado em algum commit do histórico
```

POR QUE É PERIGOSO

Apagar a chave num commit novo não a remove dos commits antigos: quem clonar o repositório encontra.

PASSO A PASSO PARA CORRIGIR

1. Revogue primeiro cada chave encontrada no histórico.
2. Só depois, se quiser, reescreva o histórico (git filter-repo).
3. Ative um verificador de segredos antes do commit (gitleaks).

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Um arquivo .env já foi commitado no passado" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

Revogue as chaves que estavam nesse arquivo na época.

AUTENTICAÇÃO E AUTORIZAÇÃO

CRÍTICO SS-03 · item 2.2

Senha guardada ou comparada sem hash forte

Possível senha em texto puro ou com hash rápido (SHA-256 sem sal).

ONDE ESTÁ

```
src/app/api/login/route.ts:6 · comparação direta de senha
if (!user || user.password !== body.password) return Response.json({ error:
"inválido" }, { status: 401 });
```

POR QUE É PERIGOSO

As senhas estão em texto puro no banco. Se o banco vazar, o atacante entra na conta dos seus usuários aqui e em outros serviços onde eles repetem a senha.

PASSO A PASSO PARA CORRIGIR

1. Instale o argon2 (npm i argon2).
2. No cadastro, salve await argon2.hash(senha) no campo password.
3. Em src/app/api/login/route.ts, troque user.password !== body.password por !(await argon2.verify(user.password, body.password)).
4. Migre os usuários antigos gerando o hash no próximo login.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Em src/app/api/login/route.ts e no cadastro, troque o armazenamento e a comparação de senha em texto puro por argon2 (hash no cadastro, argon2.verify no login). Inclua a migração que gera o hash no próximo login de cada usuário. Não altere outras rotas e mostre o diff antes de commitar.

CRÍTICO SS-04 · item 2.4

Consultas por ID sem conferir o dono do registro

3 consulta(s) buscam, alteram ou apagam por ID sem filtro de usuário ou organização por perto.

ONDE ESTÁ

```
src/app/api/pacientes/[id]/route.ts:3
const paciente = await prisma.paciente.findUnique({ where: { id: params.id } });
```

```
src/app/api/pacientes/[id]/route.ts:7
await prisma.paciente.delete({ where: { id: params.id } });
```

```
src/app/api/webhooks/stripe/route.ts:5
await prisma.assinatura.update({ where: { id: event.data.object.client_reference_id
}, data: { status: "ativa" } });
```

POR QUE É PERIGOSO

Qualquer usuário logado lê e apaga pacientes de outras clínicas trocando o ID em /api/pacientes/[id]. Como são dados de saúde, é o pior vazamento possível pela LGPD.

PASSO A PASSO PARA CORRIGIR

1. Em src/app/api/pacientes/[id]/route.ts, pegue o usuário logado no início do GET e do DELETE.
2. Troque findUnique({ where: { id } }) por findFirst({ where: { id, userId: user.id } }).
3. No DELETE, use deleteMany com o mesmo filtro e confira se apagou algo.
4. Responda 404 quando o paciente não pertencer ao usuário.
5. Teste com dois usuários: A não pode abrir nem apagar o paciente de B.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Em src/app/api/pacientes/[id]/route.ts, exija login e filtre GET e DELETE pelo usuário logado (where: { id, userId: user.id }), respondendo 404 quando o paciente não pertencer a ele. Crie um teste com dois usuários provando que um não acessa o paciente do outro. Mostre o diff antes de commitar.

Confirme se a posse é conferida em outro lugar (por exemplo, numa checagem logo depois da busca).

CRÍTICO SS-05 · item 2.5

Tabelas do Supabase sem Row Level Security

1 tabela(s) criadas nas migrations sem "enable row level security".

ONDE ESTÁ

```
supabase/migrations/001_init.sql:1
create table public.pacientes (id uuid primary key, nome text, cpf text, diagnostico
text, user_id uuid);
```

POR QUE É PERIGOSO

A tabela pacientes não tem RLS. Qualquer pessoa com a anon key (que está no front) lê nome, CPF e diagnóstico de todos os pacientes.

PASSO A PASSO PARA CORRIGIR

1. Crie uma migration com: alter table public.pacientes enable row level security;
2. Crie policies de select, insert, update e delete usando auth.uid() = user_id.
3. Aplique no projeto (supabase db push) e confira no painel em Database → Tables.
4. Teste com a anon key sem login: a consulta deve voltar vazia.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Crie uma nova migration em supabase/migrations que ative a RLS na tabela public.pacientes e crie policies de select, insert, update e delete usando auth.uid() = user_id. Mostre o SQL para eu revisar antes de aplicar.

Confirme no painel (Database → Tables) se a RLS não foi ativada manualmente por lá.

ALTO SS-09 · item 2.1

Token de segurança gerado com Math.random

Math.random é previsível: tokens de recuperação ou convite podem ser adivinhados.

ONDE ESTÁ

```
src/app/api/login/route.ts:9
const reset = Math.random().toString(36).slice(2);
```

POR QUE É PERIGOSO

Autenticação feita na mão costuma falhar em detalhes, como token de recuperação gerado com Math.random.

PASSO A PASSO PARA CORRIGIR

1. Troque por crypto.randomBytes(32).toString("hex") ou crypto.randomUUID().
2. Faça o token expirar e valer uma vez só.
3. Invalide os tokens antigos já emitidos.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Token de segurança gerado com Math.random" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

ALTO SS-10 · item 2.3

Rotas que parecem não exigir login

3 rota(s) ou ação(ões) sem checagem de usuário visível no arquivo.

ONDE ESTÁ

```
src/app/api/checkout/route.ts:3
export async function POST(req: Request) {
```

```
src/app/api/gerar/route.ts:3
export async function POST(req: Request) {
```

```
src/app/api/pacientes/[id]/route.ts:2
export async function GET(req: Request, { params }: { params: { id: string } }) {
```

POR QUE É PERIGOSO

As rotas /api/pacientes/[id], /api/gerar e /api/checkout respondem a qualquer pessoa, sem login.

PASSO A PASSO PARA CORRIGIR

1. Crie um helper getUsuario(req) que lê a sessão e lança 401 se não houver usuário.
2. Chame no início de cada rota listada.
3. Ou proteja /api/* no middleware.ts, deixando públicas só login, webhooks e health.
4. Teste chamando as rotas sem estar logado: devem responder 401.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Crie um middleware.ts que exija sessão em todas as rotas /api/*, exceto /api/login, /api/webhooks/* e /api/health. Confira que /api/pacientes/[id], /api/gerar e /api/checkout passam a responder 401 sem login. Mostre o diff antes de commitar.

ALTO SS-11 · item 2.6

Token de sessão guardado no localStorage

Qualquer script injetado na página consegue ler e roubar a sessão.

ONDE ESTÁ

```
src/components/Login.tsx:2
export function salvar(token: string) { localStorage.setItem("token", token); }
```

POR QUE É PERIGOSO

Token no localStorage é lido por qualquer script injetado; token sem expiração vale para sempre se for roubado.

PASSO A PASSO PARA CORRIGIR

1. Guarde a sessão em cookie HttpOnly, Secure e SameSite.
2. Defina expiração curta no token de acesso.
3. Invalide o refresh token no logout.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Token de sessão guardado no localStorage" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

ALTO SS-12 · item 2.7

Login sem limite de tentativas

Nenhuma biblioteca de rate limit no projeto e há rotas próprias de login ou recuperação de senha.

ONDE ESTÁ

```
src/app/api/login/route.ts
rota de login ou recuperação sem rate limit
```

POR QUE É PERIGOSO

A rota /api/login aceita tentativas ilimitadas: um script testa milhares de senhas por minuto.

PASSO A PASSO PARA CORRIGIR

1. Instale @upstash/ratelimit (funciona bem na Vercel).
2. Limite /api/login a 5 tentativas por minuto por IP e por e-mail.
3. Responda 429 quando passar do limite.
4. Aplique o mesmo na recuperação de senha.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Adicione rate limit em src/app/api/login/route.ts com @upstash/ratelimit: 5 tentativas por minuto por IP e por e-mail, respondendo 429. Mostre o diff antes de commitar.

MÉDIO SS-20 · item 2.6

Token JWT sem expiração

Um token roubado continua válido para sempre.

ONDE ESTÁ

```
src/app/api/login/route.ts:8
const token = jwt.sign({ sub: user.id }, process.env.JWT_SECRET!);
```

POR QUE É PERIGOSO

Token no localStorage é lido por qualquer script injetado; token sem expiração vale para sempre se for roubado.

PASSO A PASSO PARA CORRIGIR

1. Guarde a sessão em cookie HttpOnly, Secure e SameSite.
2. Defina expiração curta no token de acesso.
3. Invalide o refresh token no logout.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Token JWT sem expiração" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

BANCO DE DADOS E ENTRADAS

ALTO SS-13 · item 3.2

Entradas sem validação no backend

Nenhuma biblioteca de validação nas dependências.

ONDE ESTÁ

```
src/app/api/checkout/route.ts:4
const body = await req.json();
```

```
src/app/api/gerar/route.ts:4
const { texto } = await req.json();
```

POR QUE É PERIGOSO

Sem validação, o usuário manda campos extras (role, plano, preço) e o banco grava.

PASSO A PASSO PARA CORRIGIR

1. Valide o body de toda rota com schema (zod, class-validator, yup).
2. Ative whitelist para descartar campos desconhecidos.
3. Nunca repasse o body inteiro para create ou update.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Entradas sem validação no backend" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

ALTO SS-14 · item 3.4

Porta do banco publicada no docker-compose

Em uma VPS, essa porta fica acessível pela internet, mesmo com o firewall ativo.

ONDE ESTÁ

```
docker-compose.yml:5 · postgres:16
- "5432:5432"
```

POR QUE É PERIGOSO

Porta de banco aberta recebe tentativas de senha o dia inteiro. No Docker, "5432:5432" publica a porta mesmo com firewall.

PASSO A PASSO PARA CORRIGIR

1. Remova o mapeamento de portas do banco no docker-compose.
2. Se precisar acessar de fora, use 127.0.0.1:5432:5432 e túnel SSH.
3. Em banco gerenciado, restrinja por IP.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Porta do banco publicada no docker-compose" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

MÉDIO SS-21 · item 3.5

Aplicação conectando ao banco com usuário administrador

O usuário da conexão parece ser o dono do banco.

ONDE ESTÁ

```
.env:1
DATABASE_URL=postgresql://postgres.abcd:•••••@aws-0-sa-east-
1.pooler.supabase.com:5432/postgres
```

POR QUE É PERIGOSO

Se a aplicação for invadida, um usuário de banco com poder de dono permite apagar tudo.

PASSO A PASSO PARA CORRIGIR

1. Crie um usuário só para a aplicação, com leitura e escrita nos dados.
2. Rode as migrations com outro usuário.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Aplicação conectando ao banco com usuário administrador" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

No Supabase, o usuário padrão da connection string é o postgres. Prefira a API com RLS ou um usuário com permissões restritas.

MÉDIO SS-22 · item 3.6

Consultas que podem devolver senha ou token na resposta

Busca em tabela com campo de senha sem escolher os campos devolvidos.

ONDE ESTÁ

```
src/app/api/login/route.ts:5
const user = await prisma.user.findFirst({ where: { email: body.email } });
```

POR QUE É PERIGOSO

Devolver o objeto inteiro do banco vaza hash de senha, tokens e dados internos, mesmo que a tela não mostre.

PASSO A PASSO PARA CORRIGIR

1. Escolha os campos com select ou um DTO de resposta.
2. Remova campos sensíveis antes de responder.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Consultas que podem devolver senha ou token na resposta" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

INFRA E DEPLOY

ALTO SS-15 · item 4.2

URLs sem HTTPS na configuração

Dados trafegam sem criptografia nessas chamadas.

ONDE ESTÁ

```
src/app/api/checkout/route.ts:7
mode: "payment", success_url: "http://app.clinicafacil.com.br/ok",
```

POR QUE É PERIGOSO

Em HTTP, senha e cookie trafegam em texto aberto.

PASSO A PASSO PARA CORRIGIR

1. Troque URLs http:// de produção por https://.
2. Redirecione HTTP para HTTPS no servidor ou no proxy.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "URLs sem HTTPS na configuração" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

ALTO SS-16 · item 4.7

Stack trace ou modo debug exposto ao usuário

ONDE ESTÁ

```
src/app/api/gerar/route.ts:9
return Response.json({ error: err.stack }, { status: 500 });
```

POR QUE É PERIGOSO

Stack trace e mensagem do banco na resposta são um mapa para quem está tentando invadir.

PASSO A PASSO PARA CORRIGIR

1. Responda mensagem genérica no erro 500.
2. Mande o detalhe só para o log.
3. Desligue o modo debug e proteja o Swagger em produção.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Stack trace ou modo debug exposto ao usuário" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

BAIXO SS-27 · item 4.6

Front sem headers de segurança configurados

Nenhum X-Frame-Options ou Content-Security-Policy no next.config, vercel.json ou middleware.

POR QUE É PERIGOSO

Sem headers como X-Frame-Options e Content-Security-Policy, o navegador não bloqueia ataques comuns.

PASSO A PASSO PARA CORRIGIR

1. Use helmet na API (Express/NestJS).
2. Configure headers no next.config ou no vercel.json.
3. Comece a CSP em modo report-only.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Front sem headers de segurança configurados" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

APIS, PAGAMENTOS E LIMITES

CRÍTICO SS-06 · item 5.3

Webhook de pagamento sem verificação de assinatura

Qualquer pessoa pode simular um pagamento aprovado chamando essa rota.

ONDE ESTÁ

```
src/app/api/webhooks/stripe/route.ts:2
export async function POST(req: Request) {
```

POR QUE É PERIGOSO

O webhook aceita qualquer requisição. Alguém pode mandar um evento falso de checkout.session.completed e ativar a assinatura sem pagar.

PASSO A PASSO PARA CORRIGIR

1. Em src/app/api/webhooks/stripe/route.ts, leia o corpo bruto com await req.text().
2. Valide com stripe.webhooks.constructEvent(corpo, req.headers.get('stripe-signature'), process.env.STRIPE_WEBHOOK_SECRET).
3. Responda 400 se a assinatura for inválida.
4. Guarde o event.id e ignore eventos repetidos.
5. Teste com a Stripe CLI: stripe listen --forward-to localhost:3000/api/webhooks/stripe.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Em src/app/api/webhooks/stripe/route.ts, valide a assinatura do Stripe com stripe.webhooks.constructEvent usando o corpo bruto (req.text()) e STRIPE_WEBHOOK_SECRET, respondendo 400 se for inválida. Torne o processamento idempotente pelo event.id. Mostre o diff antes de commitar.

CRÍTICO SS-07 · item 5.4

Valor da cobrança vindo do front

Se o valor vem da requisição, o usuário troca o preço e paga quanto quiser.

ONDE ESTÁ

```
src/app/api/checkout/route.ts:6
line_items: [{ price_data: { currency: "brl", product_data: { name: body.plano },
unit_amount: body.amount }, quantity: 1 }],
```

POR QUE É PERIGOSO

O valor do checkout vem do body. O usuário troca amount para 1 e assina o plano por um centavo.

PASSO A PASSO PARA CORRIGIR

1. Crie uma tabela fixa de planos no servidor (ex.: { pro: 'price_123' }).
2. Em src/app/api/checkout/route.ts, receba só o id do plano e use line_items: [{ price: PLANOS[plano], quantity: 1 }].
3. Recuse planos fora da tabela com 400.
4. Libere o acesso só pelo webhook verificado, nunca pelo retorno do checkout.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Em src/app/api/checkout/route.ts, pare de usar body.amount. Crie uma tabela fixa de planos com os price IDs do Stripe, receba só o id do plano, valide contra a tabela e crie a sessão com price. Mostre o diff antes de commitar.

ALTO SS-17 · item 5.2

Rotas que chamam IA paga sem cota por usuário

Um usuário ou script pode gerar uma fatura alta chamando essas rotas em loop.

ONDE ESTÁ

```
src/app/api/gerar/route.ts:1
import OpenAI from "openai";
```

POR QUE É PERIGOSO

A rota /api/gerar chama o GPT-4o sem login nem cota. Um script em loop gera uma fatura alta em poucas horas.

PASSO A PASSO PARA CORRIGIR

1. Exija login na rota.
2. Crie uma cota diária por usuário (ex.: 20 gerações no plano grátis).
3. Limite o tamanho do texto de entrada.
4. Configure um limite de gasto no painel da OpenAI.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Em src/app/api/gerar/route.ts, exija login, limite o texto de entrada a 4.000 caracteres e crie uma cota diária por usuário (20 no plano grátis), respondendo 429 quando acabar. Mostre o diff antes de commitar.

MÉDIO SS-23 · item 5.1

API sem nenhum rate limit

5 arquivo(s) de rota e nenhuma biblioteca de rate limit no projeto.

POR QUE É PERIGOSO

Sem limite, qualquer script derruba a API ou consome os seus recursos pagos.

PASSO A PASSO PARA CORRIGIR

1. Aplique um rate limit global na API.
2. Use limites menores nas rotas caras.
3. Atrás de proxy, identifique o visitante pelo IP real.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "API sem nenhum rate limit" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

DADOS, LGPD E MONITORAMENTO

ALTO SS-18 · item 6.4

Dados sensíveis sem criptografia de campo

Campos com dados pessoais sensíveis: cpf, diagnostico.

ONDE ESTÁ

```
prisma.schema.prisma:8
cpf String
```

```
prisma.schema.prisma:9
diagnostico String
```

POR QUE É PERIGOSO

Dados de saúde, documentos e biometria têm proteção reforçada na LGPD; um vazamento é o pior cenário.

PASSO A PASSO PARA CORRIGIR

1. Criptografe os campos mais sensíveis com chave guardada fora do banco.
2. Restrinja o acesso por perfil.
3. Registre quem acessou cada registro.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Dados sensíveis sem criptografia de campo" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

Confirme se esses campos guardam dados sensíveis de verdade (saúde, documentos, biometria).

MÉDIO SS-24 · item 6.3

Usuário não consegue excluir a própria conta

Não encontrei rota ou ação de exclusão de conta.

POR QUE É PERIGOSO

A LGPD garante ao titular o direito de pedir a eliminação dos dados.

PASSO A PASSO PARA CORRIGIR

1. Crie a exclusão de conta: cancela assinatura, apaga arquivos e dados pessoais.
2. Anonimize o que precisa ficar por obrigação legal.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Usuário não consegue excluir a própria conta" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

MÉDIO SS-25 · item 6.5

Logs gravando senha, token ou dados pessoais

ONDE ESTÁ

```
src/app/api/login/route.ts:7
console.log("login", body.email, body.password);
```

POR QUE É PERIGOSO

Log costuma ter menos proteção que o banco e vai para serviços externos.

PASSO A PASSO PARA CORRIGIR

1. Remova logs de body, senha, token e documentos.
2. Configure redact no logger.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Logs gravando senha, token ou dados pessoais" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

BAIXO SS-29 · item 6.2

Sem política de privacidade ou termos de uso

Nenhuma página ou link de privacidade e termos encontrado.

POR QUE É PERIGOSO

A LGPD exige transparência sobre quais dados você coleta e para quê.

PASSO A PASSO PARA CORRIGIR

1. Publique a política de privacidade e os termos.
2. Coloque o link no rodapé e no cadastro.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Sem política de privacidade ou termos de uso" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

BAIXO SS-30 · item 6.6

Sem monitoramento de erros nem rota de health check

Ninguém é avisado quando o app cai ou quebra.

POR QUE É PERIGOSO

Se é o cliente que avisa que o app caiu, você já perdeu a confiança dele.

PASSO A PASSO PARA CORRIGIR

1. Crie uma rota /health.
2. Use um monitor externo com alerta no celular.
3. Capture erros com Sentry ou similar.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Sem monitoramento de erros nem rota de health check" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

USO DE AGENTES DE IA

MÉDIO SS-26 · item 7.4

Agente pode ler o .env do projeto

Nada impede o agente de ler as credenciais do .env.

ONDE ESTÁ

```
.env
arquivo de variáveis acessível ao agente
```

POR QUE É PERIGOSO

Se o agente enxerga o .env de produção, um comando errado roda contra os dados reais.

PASSO A PASSO PARA CORRIGIR

1. Adicione em .claude/settings.json: "permissions": { "deny": ["Read(./.env)", "Read(./.env.*)"] }.
2. Mantenha chaves de produção fora da sua máquina.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Agente pode ler o .env do projeto" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

BAIXO SS-31 · item 7.1

Projeto sem CLAUDE.md ou AGENTS.md

O agente não tem regras de segurança escritas para seguir.

POR QUE É PERIGOSO

Sem regras escritas, o agente repete os mesmos erros a cada sessão.

PASSO A PASSO PARA CORRIGIR

1. Crie ou complete o CLAUDE.md / AGENTS.md com regras de segurança do projeto.

PROMPT PARA O CLAUDE CODE OU CODEX CORRIGIR

Corrija o problema "Projeto sem CLAUDE.md ou AGENTS.md" nos arquivos indicados no relatório de segurança, seguindo o passo a passo. Faça a menor mudança possível e mostre o diff antes de commitar.

Verificação manual 8 ITEM(NS)

Estes pontos dependem de painéis e servidores fora do código. Confira cada um e marque.

- [ ] 1.6 **Cada chave com a menor permissão possível**
  Revise no painel de cada provedor (Stripe, GitHub, Cloudflare, AWS, Supabase) se as chaves têm só as permissões necessárias.

- [ ] 4.1 **Domínio atrás de proxy com WAF (ex.: Cloudflare)**
  Confira no painel de DNS se o domínio passa por um proxy com WAF. Em hospedagens como Vercel e Netlify, a borda já cumpre parte desse papel.

- [ ] 4.3 **Servidor com SSH só por chave e firewall fechado**
  Se você usa VPS, rode no servidor: sudo sshd -T | grep -E "passwordauthentication|permitrootlogin" e sudo ufw status. Em hospedagem gerenciada (Vercel, Render, Railway), não se aplica.

- [ ] 5.5 **Alertas e teto de gasto nos serviços pagos**
  Configure alertas de gasto na hospedagem, no provedor de IA, no banco e no serviço de e-mail.

- [ ] 6.1 **Backup automático, fora do servidor e testado**
  Confirme onde está o último backup do banco, de quando é e quando a restauração foi testada.

- [ ] 7.3 **Diff revisado antes de aceitar**
  Hábito: rode git diff --stat antes de cada commit e leia com atenção o que tocar em autenticação, pagamentos e package.json.

- [ ] 7.5 **Pacotes sugeridos pela IA conferidos**
  O projeto tem 7 dependência(s). Confira as que você não conhece antes de instalar novas.

- [ ] 7.6 **Revisão de segurança antes de cada deploy**
  Rode esta skill de novo antes de cada deploy importante.

Checklist completo 44 PONTOS

ITEM	PONTO VERIFICADO	STATUS	OBSERVAÇÃO
1.1	Chaves, senhas e tokens escritos no código	FALHA	
1.2	Arquivo .env fora do Git	APROVADO	.env ignorado pelo Git e nenhum .env versionado.
1.3	Histórico do Git sem segredos vazados	FALHA	

ITEM	PONTO VERIFICADO	STATUS	OBSERVAÇÃO
1.4	Segredos fora das variáveis públicas do front	FALHA	
1.5	Chaves separadas para desenvolvimento e produção	APROVADO	Revisado no código pelo agente.
1.6	Cada chave com a menor permissão possível	MANUAL	Revise no painel de cada provedor (Stripe, GitHub, Cloudflare, AWS, Supabase) se as chaves têm só as permissões necessárias.
2.1	Login com biblioteca consolidada, sem token previsível	FALHA	
2.2	Senhas com hash forte (argon2 ou bcrypt)	FALHA	
2.3	Toda rota da API exige login por padrão	FALHA	
2.4	Toda consulta confere o dono do recurso	FALHA	
2.5	Supabase com RLS e Firebase com regras fechadas	FALHA	
2.6	Sessão em cookie protegido e token com expiração	FALHA	
2.7	Limite de tentativas no login e na recuperação de senha	FALHA	

ITEM	PONTO VERIFICADO	STATUS	OBSERVAÇÃO
3.1	Consultas sem SQL concatenado	APROVADO	Nenhuma consulta montada por concatenação encontrada.
3.2	Toda entrada validada no backend	FALHA	
3.3	Uploads com limite, tipo conferido e armazenamento privado	NÃO SE APLICA	Revise tamanho máximo, tipo e privacidade dos uploads, se houver.
3.4	Banco sem acesso público pela internet	FALHA	
3.5	Usuário do banco da aplicação sem poder de administrador	FALHA	
3.6	Respostas da API só com os campos necessários	FALHA	
4.1	Domínio atrás de proxy com WAF (ex.: Cloudflare)	MANUAL	Confira no painel de DNS se o domínio passa por um proxy com WAF. Em hospedagens como Vercel e Netlify, a borda já cumpre parte desse papel.
4.2	HTTPS em tudo	FALHA	
4.3	Servidor com SSH só por chave e firewall fechado	MANUAL	Se você usa VPS, rode no servidor: sudo sshd -T | grep -E "passwordauthentication|permitrootlogin" e sudo ufw status. Em hospedagem gerenciada (Vercel, Render, Railway), não se aplica.
4.4	Dependências sem vulnerabilidades graves	NÃO SE APLICA	Nenhum gerenciador de pacotes encontrado.

ITEM	PONTO VERIFICADO	STATUS	OBSERVAÇÃO
4.5	CORS liberando só os seus domínios	APROVADO	Nenhuma configuração de CORS aberta encontrada.
4.6	Headers de segurança configurados	FALHA	
4.7	Erros em produção sem detalhes internos	FALHA	
4.8	Ambiente de produção separado do de testes	APROVADO	Revisado no código pelo agente.
5.1	Rate limiting nas rotas da API	FALHA	
5.2	Rotas de IA paga com cota por usuário	FALHA	
5.3	Webhooks de pagamento com assinatura verificada	FALHA	
5.4	Preço e plano decididos no backend	FALHA	
5.5	Alertas e teto de gasto nos serviços pagos	MANUAL	Configure alertas de gasto na hospedagem, no provedor de IA, no banco e no serviço de e-mail.
6.1	Backup automático, fora do servidor e testado	MANUAL	Confirme onde está o último backup do banco, de quando é e quando a restauração foi testada.
6.2	Política de privacidade e termos de uso publicados	FALHA	

ITEM	PONTO VERIFICADO	STATUS	OBSERVAÇÃO
6.3	Usuário consegue excluir a própria conta	FALHA	
6.4	Dados sensíveis com criptografia e acesso restrito	FALHA	
6.5	Logs sem senhas, tokens ou dados pessoais	FALHA	
6.6	Monitoramento de erros e de disponibilidade	FALHA	
7.1	CLAUDE.md ou AGENTS.md com regras de segurança	FALHA	
7.2	Projeto versionado no Git	APROVADO	Projeto versionado no Git.
7.3	Diff revisado antes de aceitar	MANUAL	Hábito: rode git diff --stat antes de cada commit e leia com atenção o que tocar em autenticação, pagamentos e package.json.
7.4	Agente sem acesso a credenciais de produção	FALHA	
7.5	Pacotes sugeridos pela IA conferidos	MANUAL	O projeto tem 7 dependência(s). Confira as que você não conhece antes de instalar novas.
7.6	Revisão de segurança antes de cada deploy	MANUAL	Rode esta skill de novo antes de cada deploy importante.

Quer o passo a passo completo?

O checklist gratuito e o Guia de implementação explicam os 44 pontos com exemplos de código e prompts prontos para o Claude Code.

Baixar o checklist grátis

Gerado pela skill saas-seguro · github.com/andrefelipevsilva/skill-saas-seguro · @andrevale.dev no Instagram

Varredura automatizada com revisão por IA. Não substitui um teste de invasão profissional. Este arquivo pode conter trechos sensíveis do seu código: não publique.